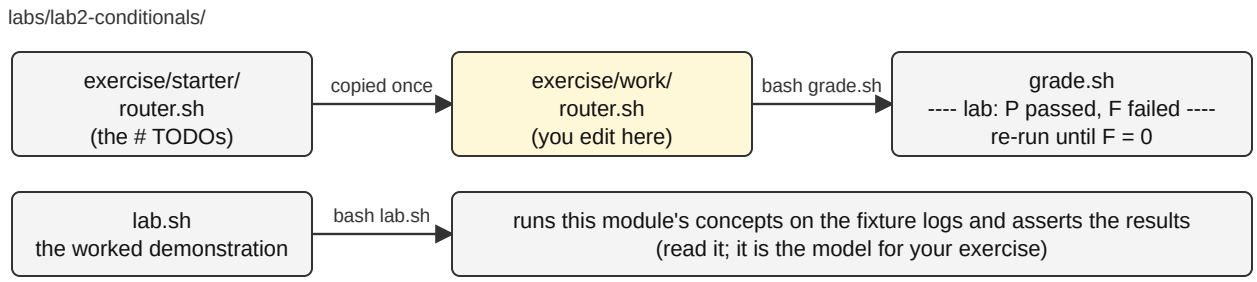
Lab 2 — Conditionals and case: Input Validation and Subcommand Routing (Instructor Version)

Instructor Note: Gray boxes and answers marked in red appear in the instructor copy only.

Topology



Environment Table

All paths are relative to `labs/lab2-conditionals/exercise/`.

File	What it is	You edit it?
`TASK.md`	The short statement of the exercise	No
`starter/router.sh`	`validate_log` (always returns 0) and `route` (prints `FIXME`)	No — the grader copies it
`work/router.sh`	**Your** copy (created by the first `grade.sh` run)	**Yes**
`grade.sh`	The grader: 7 checks. It sources your file	No
`work/REFLECTION.md`	Your two reflection answers	**Yes** (you create it)

Objectives

- **Part 1:** Prepare the exercise folder and read the grader's first report.
- **Part 2:** Build `validate_log` — succeed for a readable file, otherwise explain and fail.
- **Part 3:** Build `route` — map a subcommand name to its kind with `case`.
- **Part 4:** Verify with the grader; write the two reflection answers.

Background / Scenario

Before `logsleuth` reads a log it checks that the file exists and is readable; if not, it tells the user why and stops. And when you type `logsleuth summary` or `logsleuth bruteforce`, a `case` statement decides which handler runs. These are the two gatekeepers of the tool, and both are built on the one thing an `if` actually tests: the **exit status** of a command.

In this lab you write both in `router.sh`:

- `validate_log FILE` — returns success (0) when FILE is readable; otherwise prints `cannot read log file: FILE` to **stderr** and returns 1.

© 2026 SecMLOps · Course 0 — Bash Scripting. Course material; not for redistribution outside the course.

- `route NAME` — prints `valid` for `summary`, `top` or `errors`; `brute` for anything starting with `brute`; `unknown` for everything else.

Required Resources

- A lab session or a shell where `bash tools-check.sh` reports `ready`.
- Module M02, steps S02.3 and S02.4.
- A text editor. Documentation and AI assistance for lookups are allowed (say so in your reflection). Do **not** open `solution/`.

> **Instructor Note:** Budget 35 minutes; grading ≈ 5 minutes per learner. Observed: the starter scores `---- lab2: 1 passed, 6 failed ----` — its unconditional `return 0` passes the "readable file validates" check and fails the rest; the reference scores 7/7. A message printed to **stdout** instead of stderr still passes the grader's text check (it captures `2>&1`) but fails Question 2's reasoning — coach it.

Part 1: Prepare the exercise folder

Step 1: Run the grader once.

```
cd labs/lab2-conditionals/exercise
bash grade.sh
```

Expected: `---- lab2: 1 passed, 6 failed ----`. The one pass is an accident: the stub returns 0 for *every* file, so a readable file "validates".

Step 2: Open `work/router.sh`.

Two functions, two `# TODO` comments. The file is sourced; it only defines the functions.

Part 2: Build `validate_log`

Step 1: Test readability, then choose a channel for each outcome.

Replace the body of `validate_log` with:

```
validate_log() {
  local f="$1"
  if [[ -r "$f" ]]; then
    return 0
  else
    printf 'cannot read log file: %s\n' "$f" >&2
    return 1
  fi
}
```

How it works: `[[ -r "$f" ]]` is a command whose exit status is 0 when the file is readable; `if` tests that status. On failure the *explanation* goes to **stderr** (`>&2`) — for the person — and the *verdict* goes back as the **return status** — for the program that called the function (`if`, `&&`, `||`).

Note: The message must contain the exact phrase `cannot read log file`; the grader checks for it.

© 2026 SecMLOps · Course 0 — Bash Scripting. Course material; not for redistribution outside the course.

Step 2: Try it by hand.

```
source work/router.sh
validate_log /etc/hostname; echo "status=$?"
validate_log /nope/missing.log; echo "status=$?"
validate_log /nope/missing.log 2>/dev/null; echo "status=$?"
```

Expected: `status=0`; then the message followed by `status=1`; then **no message** but still `status=1` — the explanation and the verdict travel on different channels.

Part 3: Build `route`

Step 1: Write the dispatcher.

Replace `printf 'FIXME\n'` with a `case`:

```
route() {
  case "$1" in
    summary|top|errors) printf 'valid\n' ;;
    brute*)             printf 'brute\n' ;;
    *)                  printf 'unknown\n' ;;
  esac
}
```

How it works: `case` compares the value with each **pattern** in order and runs the first match. `summary|top|errors` lists three exact words; `brute*` is a prefix pattern (the `*` is a wildcard, so it must **not** be quoted); `*` alone is the catch-all.

Step 2: Try it.

```
route summary; route bruteforce; route frobnicate
```

Expected: `valid`, `brute`, `unknown`.

Part 4: Verify and reflect

Step 1: Run the grader until it is green.

```
bash grade.sh
```

Expected: `---- lab2: 7 passed, 0 failed ----`. The checks: a readable file validates (returns 0); a missing file fails (non-zero); a missing file prints `cannot read log file`; `summary` → `valid`; `errors` → `valid`; `bruteforce` → `brute`; `frobnicate` → `unknown`.

Symptom	Cause	Fix
`bruteforce -> brute` fails, prints `unknown`	The pattern is quoted (`"brute*"`), so it is a literal word	Remove the quotes: `brute*)`
`a missing file prints cannot read log file` fails	Wording differs	Use the exact phrase `cannot read log file:`
`a missing file fails` passes but the readable check fails	The branches are swapped	`-r` true → `return 0`

© 2026 SecMLOps · Course 0 — Bash Scripting. Course material; not for redistribution outside the course.

Step 2: Reflection.

Create `work/REFLECTION.md` and answer in your own words (3–5 sentences each):

Question 1: `if` works with `grep`, with `[[ ]]`, and with your own functions. What is the *one* thing `if` actually tests?

> **Answer:** **Met:** `if` tests the *exit status* of the command that follows it (0 → the `then` branch). **Strong:** notes that `[[ ]]` and `grep -q` are commands with statuses, so functions that `return` can be used the same way. **Not met:** "it tests whether the condition is true" without mentioning the status.

Question 2: `validate_log` both prints a message and returns 1. Who consumes each, and what would break if you printed the message to stdout instead?

> **Answer:** **Met:** the message is for a person (stderr); the status is for programs (`&&`, `||`, `if`); printing the message to stdout would mix it with data a caller captures. **Strong:** gives the capture example (`out=$(…)` swallowing the message) or mentions that `2>/dev/null` hides only the message. **Not met:** "both tell the user it failed."
>
> *Calibration.* Met: "`return 1` lets `||` and `if` react; the stderr line explains to whoever is watching. On stdout the explanation would become part of whatever output the caller is parsing." Not met: "Stdout and stderr are both printed to the terminal so it doesn't matter." — return it with "what if the caller captures the output into a variable?".

Hand-in and grading

Hand in `work/router.sh`, `work/REFLECTION.md` and your final grader output. **Pass** = `0 failed` **and** both reflection answers name the mechanism. Unlimited attempts.

> **Instructor Note:** **Grading procedure (≈5 min per learner).**
>
> 1. `GRADE_TARGET=/path/to/their/work bash labs/lab2-conditionals/exercise/grade.sh` → require `---- lab2: 7 passed, 0 failed ----`.
> 2. Read `REFLECTION.md` against the descriptors. Record `A-03 PASS` only with 7/7 and both answers at Met or better.
> 3. **Acceptable variations:** `[[ -r "$f" ]] || { printf … >&2; return 1; }` (idiomatic, passes); `echo` instead of `printf` for the words (passes). A `route` written with `if/elif` passes but misses the lesson — ask for a `case` rewrite.
> 4. **Model feedback (Not yet):** "`route bruteforce` gave `unknown`: your `brute*` pattern is quoted — quotes make it a literal word." Remediation: S02.3.
> 5. **Equivalent reassessment:** same grader; Q1′ "Why does `[ $v = x ]` fail when `v` is empty?" (Met: `[` is a command; the empty word disappears from its argument list) and Q2′ "Why does `logsleuth` use exit 2 — not 1 — for an unknown command?" (Met: usage errors are a different class from runtime failures, so callers can tell them apart).

© 2026 SecMLOps · Course 0 — Bash Scripting. Course material; not for redistribution outside the course.

Instructor Note: Appendix — reference solution (`solution/router.sh`, observed 7/7):

```
#!/usr/bin/env bash
# Lab 2 solution — input validation and subcommand routing with [[ ]] and case.

validate_log() {
  local f="$1"
  if [[ -r "$f" ]]; then
    return 0
  else
    printf 'cannot read log file: %s\n' "$f" >&2
    return 1
  fi
}

route() {
  case "$1" in
    summary|top|errors) printf 'valid\n' ;;
    brute*)             printf 'brute\n' ;;
    *)                  printf 'unknown\n' ;;
  esac
}
```

© 2026 SecMLOps · Course 0 — Bash Scripting. Course material; not for redistribution outside the course.